

OOP 練習 9：例外類別

學習目標

- 會定義 `ZCX_` 自訂例外類別，理解 `CX_STATIC_CHECK` vs `CX_DYNAMIC_CHECK` 的選擇
- 會在方法簽章宣告 `RAISING`、用 `RAISE EXCEPTION TYPE` 丟例外
- 會寫 `TRY / CATCH ... INTO / ENDTRY`，從例外物件取出訊息 (`get_text`) 與現場資料
- 理解 `CLEANUP` 的定位：例外往外傳時的善後區，不是處理例外的地方
- 能對照 ex15 的 classic `EXCEPTIONS` + `sy-subrc`，說出類別型例外好在哪

事前準備

ADT 建立兩個類別 + 一支測試程式，套件 `$TMP`：`ZCX_0009_NO_FLIGHT`（例外類別）、`ZCL_0009_FLIGHT_READER`、程式 `ZR_0009_<你的姓名縮寫>`。

題目需求

情境：查詢某航空公司的航班（`SFLIGHT`），查無資料時不再回傳空表格默默帶過，而是丟出「帶著現場資料」的例外，強迫呼叫端面對。

1. 例外類別 `ZCX_0009_NO_FLIGHT`，繼承 `CX_STATIC_CHECK`：
 - 公開屬性 `mv_carrid TYPE s_carr_id READ-ONLY`——例外物件帶著出事的航空公司代碼
 - `constructor`：`IMPORTING iv_carrid`（記得先 `super->constructor( )`）
 - `get_text` REDEFINITION：回傳人看得懂的錯誤訊息
2. `ZCL_0009_FLIGHT_READER`，方法 `get_flights`：
 - `IMPORTING iv_carrid`、`RETURNING` 航班表格、`RAISING zcx_oo09_no_flight`
 - `SELECT SFLIGHT, sy-subrc <> 0` 時 `RAISE EXCEPTION TYPE zcx_oo09_no_flight`
`EXPORTING iv_carrid = iv_carrid`
3. 測試程式（`PARAMETERS p_carrid` 預設 `XX`）：
 - `TRY` 包住 `get_flights`，正常路徑輸出前 5 筆航班
 - `CATCH zcx_oo09_no_flight INTO DATA(lx...)`：輸出 `get_text( )` 與 `mv_carrid`
 - 第二段：巢狀 `TRY` 示範 `CLEANUP`——內層只有 `CLEANUP` 沒有 `CATCH`，例外穿過內層被外層接住，觀察 `CLEANUP` 先執行
4. 實驗（看完還原）：把 `CATCH` 整段拿掉——`CX_STATIC_CHECK` 的例外不接就編譯不過；把例外類別改繼承 `CX_DYNAMIC_CHECK` 再拿掉 `CATCH`——編譯過了，但執行期直接 dump

預期輸出（範例，`p_carrid = XX`）

錯誤：航空公司 XX 查無航班資料
(例外物件屬性 `mv_carrid = XX`)

`CLEANUP`：內層善後（例外正要往外傳）
外層接住：航空公司 ZZ 查無航班資料

(`p_carrid` 給 `AA` 可看正常路徑。)

團隊實務備註

- 團隊規範：自訂例外統一繼承 `CX_STATIC_CHECK` 或 `CX_DYNAMIC_CHECK`。選擇準則——呼叫端有能力也應該處理的用 `STATIC`（編譯器幫你盯）；呼叫端通常無能為力的程式錯誤（如參數傳錯）用 `DYNAMIC`
- 對照 ex15 classic `EXCEPTIONS`：那套靠呼叫端自覺檢查 `sy-subrc`，忘了就默默往下跑；類別型例外不宣告不處理直接編譯不過，而且例外物件能帶任意資料、能一路往外傳不用每層轉手
- `CLEANUP` 只做善後（釋放鎖、回復狀態），不要在裡面「吞掉」或處理例外
- 新程式一律用類別型例外；只有呼叫舊 FM 時才會碰到 classic `EXCEPTIONS`（接到後轉丟 `ZCX_` 是常見包法）

思考題

1. `get_flights` 查無資料，「回傳空表格」跟「丟例外」各在什麼情境合理？（提示：空結果是不是「正常業務狀況」——查詢畫面 vs 計價引擎）
2. 例外類別的 `mv_carrid` 為什麼設 `READ-ONLY` 而不給 `setter`？（提示：例外是「事發當下的快照」）
3. `TRY` 裡呼叫了兩個都會丟 `zcx_oo09_no_flight` 的方法，`CATCH` 接到後怎麼知道是誰丟的？（提示：例外物件還能帶什麼？`previous` 參數是做什麼用的？）

答案

見 `zcx_oo09_no_flight.clas.abap`、`zcl_oo09_flight_reader.clas.abap` 與 `zr_oo09_exception_demo.prog.abap`（SAP 端程式 `ZR_0009_EXCEPTION_DEMO`）。